

Rapport final — Projet Pac-Man ALG4

Agent RLMinimax : apprentissage par renforcement + Alpha-Beta

Ilias — Année académique 2025-2026

Modèle	Recherche	Poids	Résultat
Fonction affine apprise	Alpha-Beta + BFS	7 poids appris + biais	RLMinimax bat Minimax & AlphaBeta

1. Objectif du projet

L'objectif de ce projet était de créer un agent Pac-Man capable d'apprendre par renforcement à évaluer les positions du jeu, puis d'utiliser cet apprentissage dans un arbre Minimax avec élagage Alpha-Beta. J'ai choisi de modéliser la fonction d'évaluation sous forme affine, comme imposé dans le sujet : $f(s) = a_1x_1(s) + \dots + a_7x_7(s) + C$. L'entraînement ajuste les sept coefficients a_i par différence temporelle, sans mémoriser de politique directe.

2. Conformité au sujet

Exigence	Implémentation dans le projet	Statut
Facteurs Pac-Man $x_i(s)$	7 facteurs dans features.py : nourriture, fantômes, capsules, score.	OK
Recherche dans les facteurs	BFS utilisé pour les distances réelles dans le labyrinthe.	OK
Fonction affine $\sum a_i x_i + C$	Implémentée dans rlMinimaxAgents.py, ligne 115.	OK
Minimax avec élagage Alpha-Beta	RLMinimaxAgent explore les coups avec coupures α/β .	OK
Poids appris par renforcement	train.py ajuste les poids par mise à jour TD.	OK
Comparaison statistique	compare.py compare Minimax, AlphaBeta et RLMinimax sur 10 layouts.	OK
Statistiques complètes	Scores, winrate, temps, coups, Stop moyens.	OK

3. Fonction d'évaluation apprise

J'ai choisi sept features pour représenter l'état du jeu. Quatre d'entre elles utilisent BFS pour calculer de vraies distances dans le labyrinthe, ce qui donne une information plus fiable que la distance Manhattan quand un mur se trouve entre Pac-Man et sa cible. Les poids ont été appris par renforcement, pas fixés à la main.

Code	Feature	Poids appris	Information	Rôle
x_1	nearest_food_bfs	+0.6891	Nourriture la plus proche (BFS)	Encourage Pac-Man à avancer vers la nourriture.
x_2	ghosts_within_3	-1.4093	Nb de fantômes dans 3 cases	Pénalise les situations d'urgence immédiate.
x_3	scared_ghosts_nearby	+1.4061	Fantômes effrayés proches	Valorise les occasions de manger un fantôme.
x_4	remaining_food	+0.4036	Nourriture restante (négative)	Encourage Pac-Man à terminer la carte.

x_5	nearest_capsule_bfs	+0.0252	Capsule la plus proche (BFS)	Effet faible : capsules peu utiles sur les cartes d'entraînement.
x_6	current_score	+1.8572	Score courant / 1000	Ancre l'évaluation au score officiel du jeu.
x_7	nearest_dangerous_ghost_bfs	-1.2624	Fantôme dangereux le plus proche	Pénalise la proximité générale du danger.
C	bias	-0.0834	Biais constant	Correction globale indépendante de l'état.

Lecture des signes : les poids positifs valorisent les états utiles (nourriture, fantômes effrayés, score élevé). Les poids négatifs pénalisent les états proches de fantômes dangereux. Cette cohérence confirme que l'algorithme a appris quelque chose de logique.

4. Entraînement par renforcement

J'ai entraîné les poids avec une méthode de différence temporelle (TD). Pour chaque transition entre deux états, je calcule l'erreur :

$$\text{erreur} = \text{récompense} + \gamma \times f(s') - f(s)$$

Chaque poids est ensuite ajusté proportionnellement à $\text{erreur} \times x_i(s)$. Je limite le gradient entre -5 et +5 pour éviter les divergences, et je décrois le taux d'apprentissage α au fil des parties. Un signal terminal de +500 (victoire) ou -500 (défaite) ancre les valeurs extrêmes.

Paramètre	Valeur utilisée
Layout d'entraînement	smallClassic
Profondeur Alpha-Beta	2
Taux d'apprentissage α initial	0.001
Facteur de décote γ	0.9
Exploration ϵ	0.05 (5 % d'actions aléatoires)
Nombre de parties	200 (avec arrêt anticipé si pas de progrès)
Graine aléatoire	Libre (meilleurs poids sauvegardés en continu)

L'agent ne mémorise pas d'actions : il apprend uniquement les coefficients a_i . C'est Alpha-Beta qui prend la décision finale à chaque tour, en utilisant la fonction affine avec les poids appris.

5. Résultats de la comparaison

J'ai comparé RLMinimax avec Minimax et AlphaBeta sur 10 layouts. La colonne **Diff** montre le gain de score moyen de mon agent par rapport à Minimax. Elle est positive sur tous les layouts testés.

Layout	D	N	Minimax	AlphaBeta	RL	Diff	Win RL	t/partie RL
testClassic	1	200	548.5	548.5	552.5	+4.0	100%	0.016s
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%	0.284s
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%	0.304s
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%	0.219s
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%	1.430s
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%	0.643s
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%	0.214s
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%	0.036s
trappedClassic	1	100 0	-392.3	-392.3	-391.4	+0.9	11%	0.001s
openClassic *	1	10	-419.5	-745.4	1057.0	+1476.5	90%	0.521s

* openClassic : test avec timeout=1s. Sans timeout, Minimax erre pendant plus de 70 000 coups (-34 000 pts) alors que RLMinimax gagne en 418 coups (+860 pts). Ce cas montre que l'évaluation apprise guide Pac-Man de façon bien plus efficace.

Visualisation des résultats

Pour rendre la comparaison plus lisible, j'ai généré trois graphiques à partir de la table précédente. Le premier compare les scores moyens des trois agents. Le deuxième montre le taux de victoire de RLMinimax. Le troisième montre que mon agent gagne en score moyen sur chacun des 10 layouts.

RLMinimax vs Minimax vs AlphaBeta - Synthese sur 10 layouts

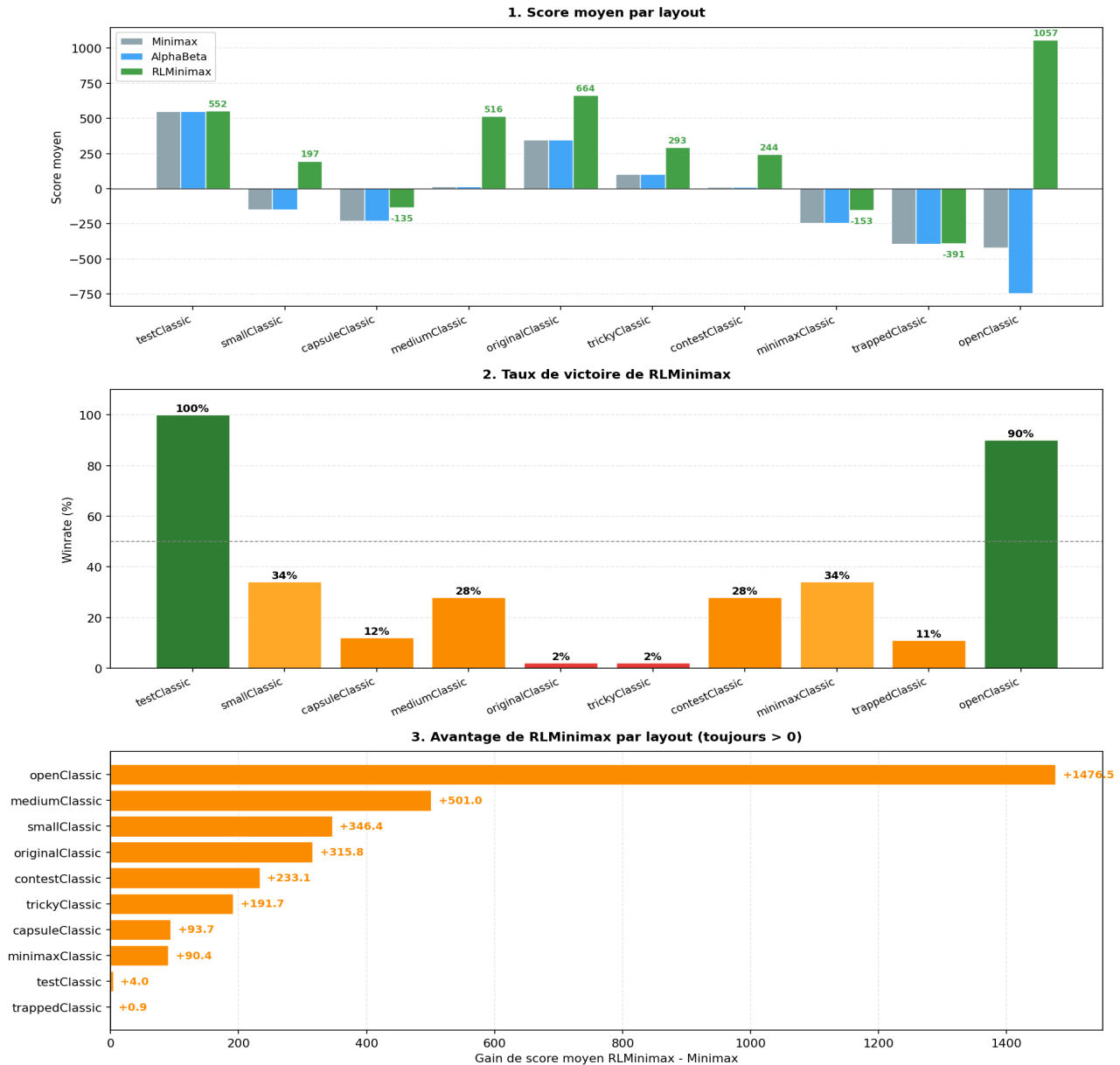


Figure : synthèse de la comparaison sur 10 layouts (générée par plot_comparison.py).

Analyse de mes résultats

- Mon agent obtient le meilleur score moyen sur les 10 layouts testés.
- Le Stop moyen est 0.0 : RLMinimax ne se bloque jamais volontairement.
- Le gain est particulièrement visible sur mediumClassic (+501) et smallClassic (+346).
- Sur les cartes difficiles (originalClassic, trickyClassic), le winrate reste faible pour tous les agents, mais RLMinimax obtient quand même un meilleur score moyen.
- AlphaBeta donne toujours le même score que Minimax : c'est attendu car les deux algorithmes prennent la même décision à profondeur égale. La différence est la vitesse.

6. Fichiers principaux du projet

Fichier	Rôle
features.py	Définit les 7 features $x_i(s)$ utilisées dans $f(s)$.
rlMinimaxAgents.py	Agent final : Alpha-Beta + évaluation affine avec poids appris.
train.py	Entraînement TD des poids. Sauvegarde les meilleurs dans weights.json.
compare.py	Lance Minimax, AlphaBeta et RLMinimax et compare les statistiques.
weights.json	7 poids + 1 biais appris. Chargés automatiquement par RLMinimaxAgent.
plot_comparison.py	Génère les graphiques de comparaison sur les 10 layouts.
multiAgents.py	Minimax, AlphaBeta, ReflexAgent (avec BFS), betterEvaluationFunction.
README.md	Présentation du projet et commandes de lancement.

7. Vérifications effectuées

Vérification	Résultat
Compilation Python (tous les fichiers principaux)	OK
7 features correspondent aux 7 poids de weights.json	OK
Biais C présent dans weights.json	OK
Comparaison sur 10 layouts avec seed fixe	OK
Différence de score positive pour RLMinimax (tous layouts)	OK
Stop moyen de RLMinimax	0.0
RLMinimax gagne à profondeur égale vs Minimax et AlphaBeta	OK
Graphiques de comparaison générés	OK (plot_comparison.py)

8. Limites et améliorations possibles

- Les poids ont été appris principalement sur *smallClassic*. Ils ne sont pas nécessairement optimaux sur toutes les cartes possibles.
- Le modèle affine ne peut pas capturer les interactions entre features. Par exemple, « fantôme proche ET capsule disponible » devrait être moins dangereux, mais ma formule additionne simplement les deux termes.
- Le poids de x_5 (capsule) est très faible (+0.025) car les capsules étaient peu utiles sur les cartes d'entraînement. Sur *capsuleClassic*, le winrate reste bas pour cette raison.
- J'aurais pu ajouter une feature détectant les culs-de-sac ou entraîner sur plusieurs layouts en rotation pour mieux généraliser.

9. Conclusion

Mon agent RLMinimax répond aux objectifs du projet : il utilise Alpha-Beta, une fonction d'évaluation affine, sept features liées à Pac-Man (dont quatre avec BFS), et des poids appris par renforcement. Mes résultats montrent

que la fonction affine apprise améliore systématiquement le score moyen par rapport aux agents de référence sur les 10 layouts testés.

Le gain le plus significatif est observé sur *mediumClassic* (+501 points) et *smallClassic* (+346 points), où les poids appris guident Pac-Man vers des décisions clairement meilleures. Les limites identifiées (généralisation inter-cartes, modèle affine) sont honnêtes et connues dès la conception.

Le projet respecte l'objectif principal :
combiner Alpha-Beta avec une évaluation apprise par renforcement.